

SLO-RAG: SLO Aware Serverless RAG Platform

Problem Statement

Retrieval-Augmented Generation (RAG) systems combine pre-trained large language models with external knowledge retrieval to improve performance in knowledge-intensive applications [1]. RAG applications typically involve multiple distinct workflows: document ingestion, query retrieval and LLM inference, each with varying resource demands. These workflows are inherently triggered by external rather than running continuously, which naturally maps them to serverless computing paradigm. For example, document uploads trigger ingestion and embedding generation, while user queries trigger retrieval and generation phases. Such alignment provides a potential for achieving cost efficiency by paying only for actual inference time, rather than maintaining warm GPU instances for sporadic queries and zero idle cost means that document ingestion happens on-demand without requiring persistent compute resources [2,3].

Furthermore, the serverless execution model provides potential for dynamic model selection, enabling development of sophisticated algorithms that can choose between different LLMs on a per-query basis rather than committing to a single running model [4]. However, the stateless nature of serverless functions introduces challenges for maintaining user sessions across multiple queries. Conversation context must be saved and retrieved when needed for longer running sessions.

The architecture of RAG applications and serverless paradigm provide tunable knobs for managing trade-offs between accuracy, latency and cost. Through adjustment of retrieval strategies, selection of appropriate models and dynamic allocation of resources, RAG applications could adapt to various queries, workloads and service level objectives (SLOs).

Application Scenario

We propose a serverless RAG platform consisting of four independent workflows that demonstrate how event-driven architecture enables cost-efficient and scalable knowledge-intensive applications.

(1) Document Ingestion Workflow

This workflow is entirely asynchronous and decoupled, utilizing a central Message Broker (MQ) to pass data and trigger subsequent stages. This design maximizes throughput and enables highly parallel processing during bulk document uploads.

Stage 1: Document Upload and Text Extraction

The workflow is initiated externally when a user uploads a file to the Object Storage, which automatically generates a system event notification. This event triggers the Text Extraction Function, which retrieves the file and parses the content to extract the raw text. Upon successful extraction, the function acts as a publisher, pushing a `DOCUMENT_UPLOADED` message to the Message Broker.

Stage 2: Text Chunking and Segmentation

The process continues when the Chunking Function subscribes to and consumes the `DOCUMENT_UPLOADED` message from the broker. The function applies segmentation logic to divide the raw text into coherent, manageable chunks based on configurable size and overlap parameters. Once processing is complete, the function publishes a `CHUNKS_READY` message to the broker, passing the list of text segments and enriched metadata to the next stage.

Stage 3: Embedding Generation

The Embedding Function is triggered by the `CHUNKS_READY` message and represents the most compute-intensive phase of the pipeline. This function loads the specific machine learning model and converts each text chunk into a vector representation. To maintain the event-driven chain, the function packages the resulting vectors with their source text and attributes, publishing an `EMBEDDINGS_GENERATED` message to the broker for the final downstream consumer.

Stage 4: Vector Storage and Indexing

The final phase activates when the Vector Storage Function receives the `EMBEDDINGS_GENERATED` message. This function interfaces directly with the Vector Database, performing a bulk upsert operation to index the new embeddings alongside their document identifiers and timestamps. The successful completion of this operation signifies that the document is fully ingested and instantly searchable, allowing the entire workflow to scale back to zero until the next upload occurs.

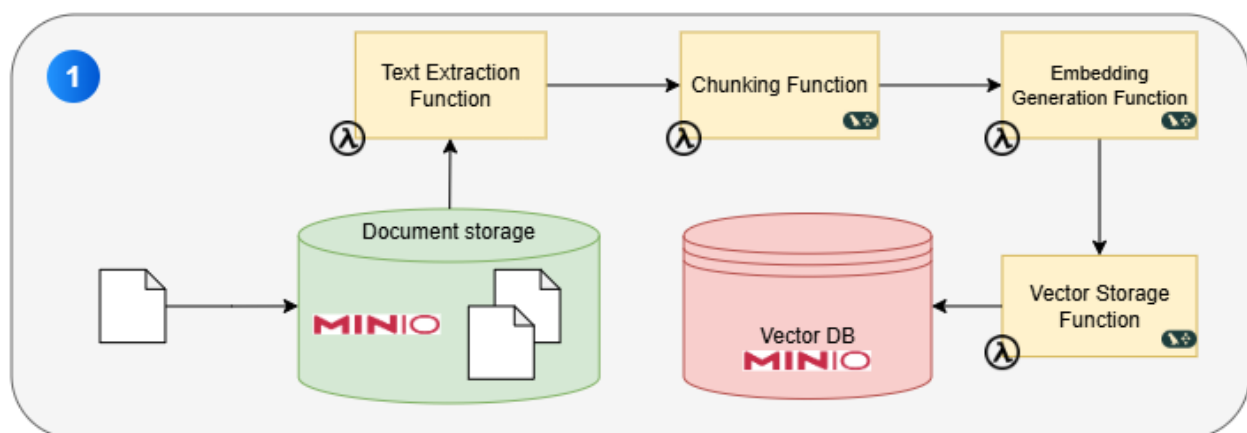


Figure 1: Document Ingestion Workflow

(2) Inference Workflow

The Inference Workflow is a highly scalable, two-stage process triggered directly by a user query, designed to dynamically allocate compute resources and select the most cost-effective Language Model (LLM) for every request.

Stage 1: Context Retrieval and Augmentation

The first phase starts with generating a vector embedding of the user's prompt. This embedding is then used to search the pre-indexed vector database for relevant document chunks. This phase scales independently based on retrieval complexity: simple queries require minimal compute time to return a few key chunks, while complex queries may require extensive, resource-demanding searches. The serverless model allows this phase to allocate compute resources only as needed without fixed provisioning. The process concludes by assembling the augmented prompt consisting of retrieved relevant document chunks, original user's prompt and previous conversational history.

Stage 2: Response Generation and Dynamic Routing

The second phase is the response generation phase. In this phase, the augmented prompt is passed to the SLO-aware Router that manages the trade-offs between accuracy, latency, and cost. The router analyzes the query's complexity by utilizing a pretrained classifier model and decides which LLM to use for inference. Available models include:

- Locally deployed [Llama-3.1-8B-Instruct](#): Hosted on serverless GPU containers (e.g., via vLLM), provides the lowest per-query cost but requires evaluating the risk of high cold start latency and potential accuracy limitations.
- Remotely deployed [Llama-3.3-70B-Instruct](#): Hosted on [Groq](#) platform results with higher per-query cost but offers low initialization overhead and reliable baseline accuracy, making it the choice for meeting strict latency or high-accuracy.
- Proprietary LLMs: Invoked over API calls provide best accuracy with medium latency and high per-query costs.

Router utilizes a fine-tuned DistilBERT classifier. The classifier outputs complexity prediction with a confidence score (0.0 to 1.0). If the score exceeds a configurable confidence threshold, the request is routed to the remote Llama-70B or proprietary APIs, otherwise, it is processed by the cost efficient local Llama-8B instance. The threshold acts as a tunable knob, allowing us to dynamically manage accuracy, cost and latency trade-offs based on SLO requirements. The chosen LLM generates the final response, which is then immediately returned to the user. Critically, the completion of this process publishes an INFERENCE_COMPLETED event to the Message Broker. This event payload includes the session_id, original query, and final response.

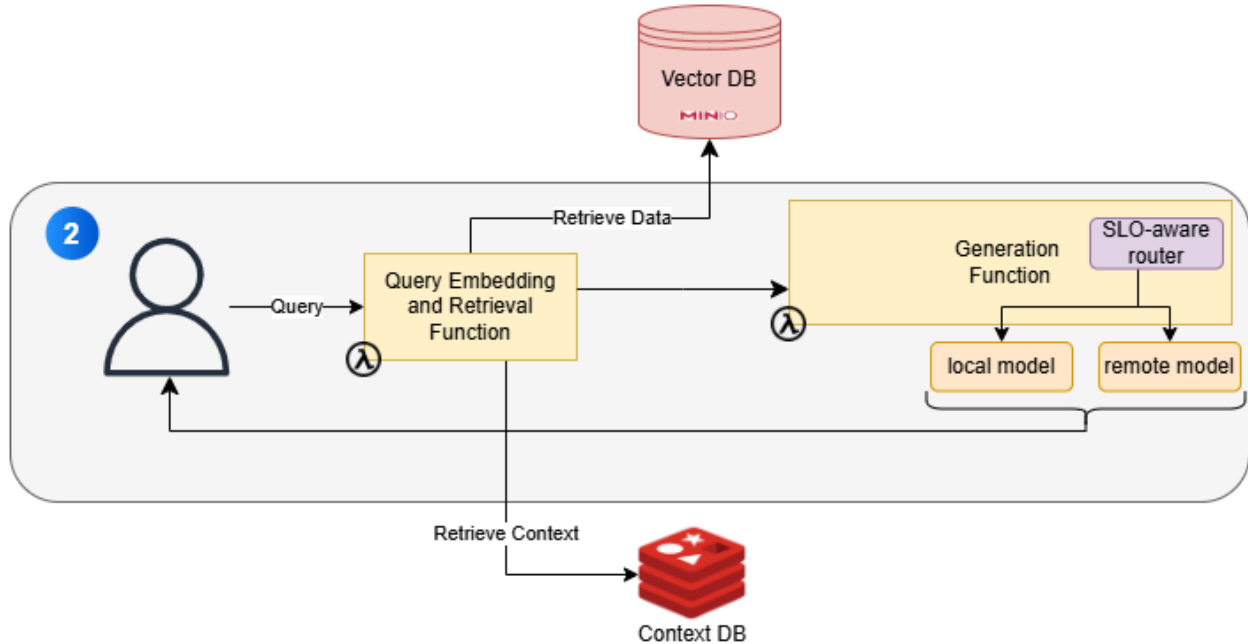


Figure 2: Inference Workflow

(3) Conversational State Management Workflow

Stage 1: Context Retrieval (Read Operation)

This stage is triggered immediately when the Query Handler Function receives a new user request. Before any processing occurs, the function extracts the `session_id` from the request header and queries the Key-Value Store to retrieve the associated conversation history. This retrieved context is then concatenated with the new query to form a contextualized prompt, which is passed downstream to the retrieval and generation components. If no session exists, a new list is initialized.

Stage 2: State Persistence (Write Operation)

This stage acts as a "side-effect" of the inference process, ensuring strictly decoupled execution. Once the Generation Function produces a final response, it publishes an `INFERENCE_COMPLETED` event to the Message Broker. The State Manager Function subscribes to this event, receiving a payload that includes the `session_id`, the original user query, and the generated response. It then performs an append operation to the Key-Value storage identified by the session key. To prevent infinite memory growth, we will configure a Time-To-Live (TTL) on these keys (e.g., 24 hours), ensuring that abandoned sessions automatically expire and free up storage resources.

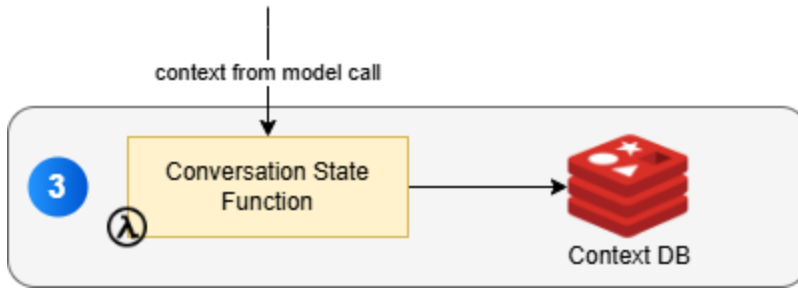


Figure 3: Conversational State Management Workflow

(4) Context Summarization Workflow

Stage 1: Limit Detection and Event Trigger

The workflow employs an event-driven trigger initiated by the State Manager Function whenever a new message is appended to a user's session history in the Key-Value Store. The State Manager performs a token count or message limit check on the current session, and if the history exceeds the pre-defined threshold, it immediately publishes a lightweight `CONTEXT_LIMIT_REACHED` event to the Message Broker. This approach ensures the summarization process is only invoked when necessary, preventing wasted compute cycles on short, inactive, or low-volume sessions. This completes the asynchronous triggering phase, decoupling the resource-heavy summarization from the user's active session.

Stage 2: Semantic Compression and State Replacement

The Context Compressor Function is triggered by and consumes the `CONTEXT_LIMIT_REACHED` event, initiating a crucial, two-part integrity process designed to prevent memory overload. First, the function executes semantic compression, retrieving the full conversation history from the Key-Value Store. It then utilizes specialized library logic to distill the oldest messages into a single, concise, narrative paragraph using a Small Language Model (SLM). Second, the function executes state replacement by performing an atomic update against the Key-Value Store: it removes the original range of detailed messages and prepends the newly created summary as a "System Message." This atomic swap is crucial for preserving data integrity and ensuring the context remains coherent while achieving the goal of reducing both storage size and future token consumption.

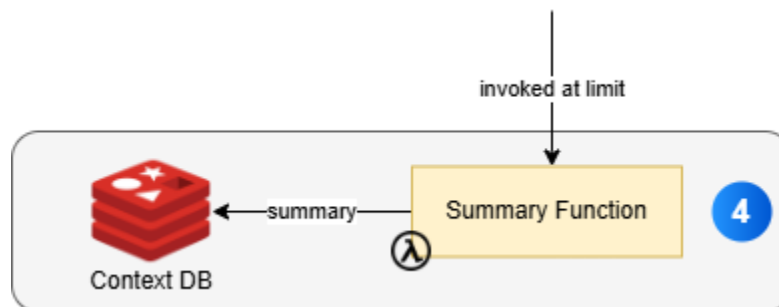


Figure 4: Summarization Workflow

Technical Details

The proposed platform adopts a decoupled, event-driven serverless architecture running on faasd, designed to optimize the trade-off between operational cost and retrieval latency. By separating the system into asynchronous document embedding services and synchronous inference services, we ensure that compute resources are only provisioned when active.

Key architectural decisions include:

- Asynchronous document ingestion workflow - triggered by MinIO storage events when a user uploads a textual document.
- Externalized state management - using LangChain/LangGraph with Redis persistence.
- SLO-Aware model routing - dynamic routing layer using classifier model for query complexity in real-time, dispatching requests to the most cost-effective model

Table 1. describes core technologies that will be used for realizing the prototype of the described system:

Category	Role in Architecture	Technology
Serverless Compute	Core Serverless Platform	faasd
Storage/State	Object Storage (Data Lake)	MinIO
	Vector Database	MinIO
	State Store & Caching	Redis
AI Orchestration	Application Framework	LangChain
Inference & Models	Local Inference Engine	vLLM
	Remote Inference API	Groq
	Local LLM	Llama-3.1-8B-Instruct

	Remote LLM	Llama-3.3-70B-Instruct
Communication	Message Broker	Kafka

Table 1. Technologies for implementing the application prototype

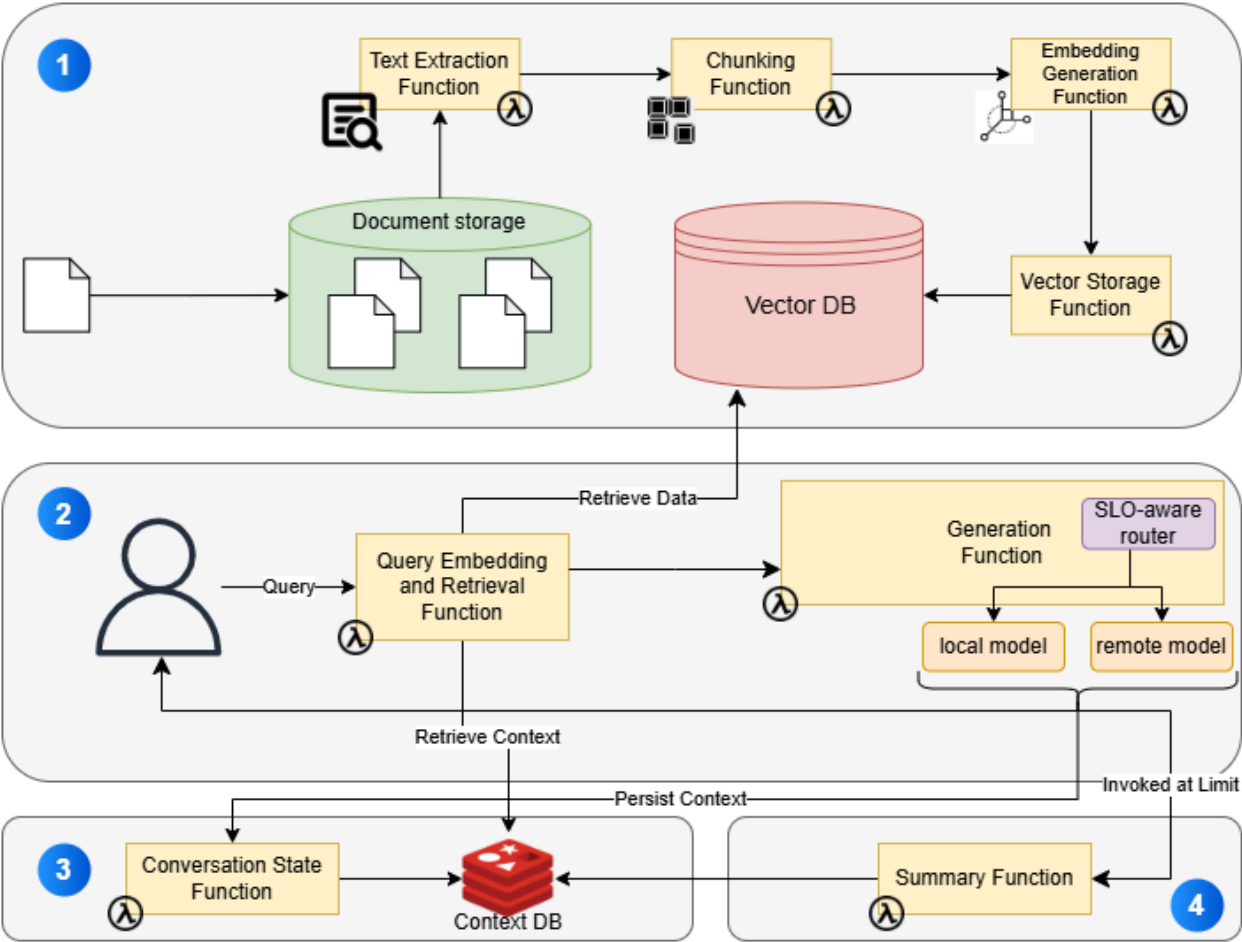


Figure 5: Serverless RAG Workflows

Experiments

Dataset

We intend to use the [SQuAD 2.0](#) dataset from Stanford which has context of the question and the ground truth. In order to add context to the dataset we will preprocess with another labeling LLM in order to obtain the context, the ground truth and the label all in one dataset.

Experiment 1: Baseline LLM Monolithic System

This experiment establishes the performance baseline by evaluating a standard **LLM-only solution** running on traditional cloud infrastructure without RAG capabilities. We will use the same SQuAD 2.0 derived workload used in subsequent RAG experiments and measure:

- **End-to-End Latency**
- **Cost per Request**
- **Output Accuracy** (evaluating against the SQuAD ground truth without retrieval). The results will quantify the inherent performance and cost of a single, non-RAG, non-serverless model, allowing for a clear comparison of the trade-offs introduced by our approach.

Experiment 2: Document Ingestion Scalability

This experiment focuses solely on evaluating the performance and cost efficiency of the **Document Ingestion Workflow (WF 1)** under varying loads. We will define different workload levels (Workloads = {1, 2, 5, 10, 15, ...} files/second) composed of various file types. By stressing the multi-stage, serverless pipeline (Extraction -> Chunking -> Embedding -> Vector Storage), we will measure the **system throughput** and **parallel processing efficiency** under high load. We want to demonstrate the viability of using a serverless architecture to achieve high-volume data ingestion.

Experiment 3: SLO-Aware Router and System Metrics

This is the core evaluation of the SLO-RAG platform's intelligence and serverless viability. We will utilize the preprocessed SQuAD dataset to stress the system. We will measure:

- **SLO Compliance Rate:** The percentage of queries that successfully meet the predefined latency and cost targets.
- **Router Accuracy:** How correctly the SLO-Aware Router selects between the cost-efficient local model (Llama-3.1-8B) and the high-performance remote models (Llama-3.3-70B or Proprietary LLMs) based on the query's complexity.
- **Cold Start Duration:** Quantifying the average duration of cold starts for the locally deployed Llama-3.1-8B model to precisely understand the cost-latency trade-off.
- **Storage Savings:** By comparing the size of conversation history before and after compression by the **Context Summarization Workflow (WF 4)**.

Key Performance Indicators and Service Level Objectives

The platform will be evaluated across two critical dimensions: task performance (including output, routing, and retrieval accuracy) and system performance (including costs, latency, and throughput). We aim to measure the effectiveness and economic viability of the serverless architecture for RAG applications.

Specifically, the key metrics and their targeted Service Level Objectives (SLOs) are defined as:

- **Latency:** The end-to-end tail latency (P95, the time from user prompt to final response) across various use cases should be less than **5 seconds**. We will also measure **cold start duration** to quantify its impact on overall system latency and understand the trade-offs between cost savings and response times.
- **Cost:** The average per-query cost is targeted to be less than **\$0.015 per request**.
- **Accuracy:** The generation and retrieval accuracy will be maintained at or above **0.8 F1 score**.
- **Throughput:** The system must demonstrate an average throughput of at least **50 queries per second**.

These performance indicators will demonstrate how the platform successfully manages trade-offs between task and system performance, ensuring SLO compliance. Additionally, document ingestion scalability will be evaluated by measuring throughput under varying upload loads and parallel processing efficiency. Finally, storage savings from the context summarization workflow will be quantified by comparing conversation history sizes before and after compression to demonstrate its effectiveness in reducing long-term storage costs.

References

- [1] Lewis, Patrick, Ethan Perez, Aleksandra Piktus, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." arXiv:2005.11401. Preprint, arXiv, April 12, 2021. <https://doi.org/10.48550/arXiv.2005.11401>.
- [2] Fu, Yao, Leyang Xue, Yeqi Huang, et al. "ServerlessLLM: Low-Latency Serverless Inference for Large Language Models." arXiv:2401.14351. Preprint, arXiv, July 25, 2024. <https://doi.org/10.48550/arXiv.2401.14351>.
- [3] Serverless retrieval augmented generation (RAG) on AWS - Aws Startups. Accessed November 4, 2025. <https://aws.amazon.com/startups/learn/serverless-retrieval-augmented-generation-on-aws>.
- [4] Jain, Kunal, Anjaly Parayil, Ankur Mallick, et al. "Intelligent Router for LLM Workloads: Improving Performance Through Workload-Aware Load Balancing." arXiv:2408.13510. Preprint, arXiv, January 7, 2025. <https://doi.org/10.48550/arXiv.2408.13510>.